

# Warm up

In words, what does `mystery` return? In what cases might it cause a problem? How would you mitigate this problem?

```
int mystery(int* arr) {  
    int i = 0;  
    while (arr[i++]) { /* intentionally empty */ }  
    return i - 1;  
}
```

# Warm up

In words, what does `mystery` return? In what cases might it cause a problem? How would you mitigate this problem?

```
int mystery(int* arr) {  
    int i = 0;  
    while (arr[i++]) { /* intentionally empty */ }  
    return i - 1;  
}
```

- Returns the index of the first zero value in `arr`
- The `while` loop iterates until it encounters a zero value in the array
- Risks going past the end of the array if there is no zero value
  - Pass array length as additional parameter and check that `i` is within bounds before accessing `arr[i]`

# Warm up

Corrected version of mystery:

```
int mystery(int* arr, int length) {  
    int i = 0;  
    while (i < length && arr[i]) {  
        i++;  
    }  
    /* use -1 to signify no zero */  
    return i == length ? -1 : i;  
}
```

- `a == b ? x : y` is a one-line conditional statement (called a *ternary operator*), which evaluates to `x` if `a == b` is true and `y` otherwise

# Precept 7: More Pointers, Arrays and Strings

Polly Ren

COS 217: Introduction to Programming Systems  
Spring 2026  
Princeton University

February 17, 2026

# Overview

1 Review of Pointers and Arrays

2 Strings

3 The `const` qualifier

4 Summary

# Relationship between pointers and arrays

- An array name is a pointer to the first element of the array
- For an array `arr`, `arr` is equivalent to `&arr[0]`

$$\text{arr} \equiv \&\text{arr}[0]$$

$$\iff \text{arr} + i \equiv \&\text{arr}[0] + i \quad \text{add } i \text{ to both sides}$$

$$\iff \text{arr} + i \equiv \&\text{arr}[i] \quad \text{pointer arithmetic}$$

$$\iff *(\text{arr} + i) \equiv *\&\text{arr}[i] \quad \text{dereference both sides}$$

$$\iff *(\text{arr} + i) \equiv \text{arr}[i] \quad * \text{ and } \& \text{ are inverses}$$

## Recall: Pointer arithmetic

- For an array name `arr`, `arr + i` is the address of element  $i$ 
  - To read or write to element at index  $i$ , use `arr[i]` or `*(arr + i)`
  - Oddly enough, `i[arr]` is also valid and equivalent (!)
    - ◊ `arr[i] = *(arr + i) = *(i + arr) = i[arr]`
- Suppose we have an array `arr` of four ints
  - Suppose the array is allocated starting at address 100, i.e., `arr = 100`
  - An `int` is 4 bytes  $\rightarrow$  the array is  $4 \times 4 = 16$  bytes in size
  - `arr + 0 = 100`  $\rightarrow$  `arr[0]` is stored at address 100
  - `arr + 1 = 104`  $\rightarrow$  `arr[1]` is stored at address 104
  - `arr + 2 = 108`  $\rightarrow$  `arr[2]` is stored at address 108
  - `arr + 3 = 112`  $\rightarrow$  `arr[3]` is stored at address 112
  - `arr + i = 100 + 4i`  $\rightarrow$  `arr[i]` is stored at address  $100 + 4i$
- True for non-array pointers: For a pointer `p` of type `type*`, `p + i` is the address of the element  $i$  elements after the one pointed to by `p`
  - The offset is calculated as  $i * \text{sizeof}(\text{type})$  bytes

# Walkthrough of rev.c, revp1.c and revp2.c

- The array name decays into a pointer to the first element of the array when used in an expression
  - Thus, `arr + i` is an address, no need to use `&`
- Replace indexing into an array with pointer arithmetic and dereferencing
  - `arr[i] → *(arr + i)`
- Can also use a sentinel pointer to traverse the array
  - `p` starts at the beginning of the array (i.e., `p = arr`)
  - Sentinel pointer can be an address one past the end of the array (i.e., `p_after_last = arr + length`)
  - `p` is incremented to move to the next element in the array (i.e., `p++` moves `p` to point to the next element in the array)
  - Keep looping while `p` has not reached the sentinel pointer using pointer comparison (i.e., `while (p != p_after_last)`)



# Arrays as function parameters

- When arrays are passed as arguments to a function, they decay into pointers to the array's first element
  - Effectively, call by reference
  - Using `sizeof` on the parameter will give the size of the pointer, not the size of the array
- This is true regardless of the parameter type used in the function signature
  - `void func(int *arr);`
  - `void func(int arr[ARRAY_LENGTH]);`
  - `void func(int arr[]);`
- Warning: compiler will not complain if the array you pass in has a different length than the one specified in the function signature
- Arrays can also serve as out-parameters, which allows functions to modify the contents of the array passed in

# Multidimensional arrays

- A multidimensional array is an array of arrays
- Example: `int arr[3][4]` can be thought of as an array of three arrays, where each inner array has four ints
- The elements of the inner arrays are stored contiguously in memory in row-major order
  - The first row (i.e., `arr[0][0]`, `arr[0][1]`, `arr[0][2]`, `arr[0][3]`) is stored first, followed by the second row, and then the third row

2d array:

|        |        |        |        |
|--------|--------|--------|--------|
| [0][0] | [0][1] | [0][2] | [0][3] |
| [1][0] | [1][1] | [1][2] | [1][3] |
| [2][0] | [2][1] | [2][2] | [2][3] |

memory layout:

|        |        |        |        |        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| [0][0] | [0][1] | [0][2] | [0][3] | [1][0] | [1][1] | [1][2] | [1][3] | [2][0] | [2][1] | [2][2] | [2][3] |
|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|

- To access the element at row `i` and column `j`, we can use `arr[i][j]` (or equivalently, `*(*(arr + i) + j)`)

# Overview

- 1 Review of Pointers and Arrays
- 2 Strings
- 3 The `const` qualifier
- 4 Summary

# Strings as character arrays

- C has no built-in string type
- We can instead represent strings as an array of `chars`
- How should we keep track of the length of the string?
  - We could store the length in a separate variable
    - ◇ Have to remember to update length variable when we modify the string
    - ◇ Cannot operate on the string without access to the length variable
  - Package the length and the `char` array together in a `struct`
    - ◇ Lots of external libraries do this
    - ◇ Additional overhead of storing the length and accessing it when we want to operate on the string
  - Somehow mark the end of the string in the array itself
    - ◇ This is the approach built into C: use a sentinel (the NUL character, `'\0'`) to signal the end of the string without external metadata
    - ◇ Standard string library functions (e.g., `strlen`) rely on presence of NUL character (and your Assignment 2 implementation should too)

# Manipulating strings

- Allocating memory for strings: `char string[100];`
  - Stores the string in an array on the stack
  - Can store strings of length up to 99 characters
- Initialising strings: `char string[] = "Hello, world!";`
  - Warning: silently truncates string if it exceeds the array size
- Computing string length: `strlen(string)`
  - `sizeof(string)` will give the size of the array allocated for string
- Modifying strings: `string[0] = 'h';`
  - Cannot modify string literals in read-only memory
    - ◇ `char* str = "Hello"; str[0] = 'h';` causes a bus error at runtime
    - ◇ But, `char str[] = "Hello"; str[0] = 'h';` is fine because `str` is an array allocated on the stack, not as a pointer to read-only memory
  - Allows read-only strings to be shared across the program without worrying about accidental modification
  - `strcpy(dest, src)` copies the string `src` to the array `dest`

# Manipulating strings even more

- Concatenating strings: `strcat(dest, src)`
- Comparing strings: `strcmp(str1, str2)`
  - Negative if `str1` is lexicographically less than `str2`
  - Positive if `str1` is lexicographically greater than `str2`
  - Zero if the strings are equal (in content and length)
- Warning: built-in string functions (e.g., `strcpy`, `strcat`) do not perform bounds checking, which can lead to buffer overflows if the destination array is not large enough to hold the resulting string
  - Always ensure that the destination array has enough space for the resulting string (including the NUL character)
  - Consider using safer alternatives (e.g., `strncpy`, `strncat`)
- See *Manipulating C Strings* handout for more details and examples

# Overview

- 1 Review of Pointers and Arrays
- 2 Strings
- 3 The `const` qualifier
- 4 Summary

# What is const and why use it?

- const indicates that a variable's value cannot be modified
- Example: `const int x = 5;` declares a constant integer variable x with value 5
  - Attempting to modify x (e.g., `x = 10;`) causes a compile-time error
- Why use const:
  - Prevents accidental modification of variables that should not be changed
  - Provides self-documentation to indicate that a variable is intended to be immutable
    - ◇ Can serve as a promise to the caller that the function will not modify the argument passed in
  - Compilers can also make certain optimisations
  - For strings, using `const char*` as the function parameter type allows the function to accept string literals without a compiler warning

warning: passing 'const char \*' to parameter of type 'char \*'  
discards qualifiers [-Wincompatible-pointer-types-discards-qualifiers]

    - ◇ Conversely, passing `char*` to a function expecting `const char*` is fine



# Promises can be broken

- `const` is a compile-time check, not a runtime guarantee
- It is still possible to modify a `const` variable through pointer manipulation or by type casting

```
const int x = 5;
int* xp = (int*)&x; /* cast away const-ness */
*xp = 10;           /* modify x through xp */
```

- But, trying to modify a variable in read-only memory (e.g., a string literal) will cause a bus error or segfault at runtime

```
const char* str = "Hello, world!";
char* p = (char*)str; /* cast away const-ness */
*p = 'h';             /* attempt to modify string literal */
```

- For the record, I am not encouraging this, just that it is possible ;)

# Pointers to constants and constant pointers

- `const` can be used in different ways with pointers
- Pointer to constant: `const int* p`
  - `p` is a pointer to an integer that cannot be modified through `p`
  - `p` itself can be updated to point to a different integer
  - (“Protects” the data)
- Constant pointer: `int* const p`
  - `p` is a pointer that cannot point to a different integer
  - The integer it points to can be modified through `p`
  - (“Protects” the pointer)
- Constant pointer to constant: `const int* const p`
  - `p` is a pointer that cannot point to a different integer
  - The integer it points to also cannot be modified through `p`
  - (“Protects” both the pointer and the data)

# Overview

- 1 Review of Pointers and Arrays
- 2 Strings
- 3 The `const` qualifier
- 4 Summary

# Summary

- Review of pointers, arrays and their relationship
- Strings are `char` arrays with a sentinel NUL character
- `const` qualifier indicates immutability and can be used in different ways with pointers
- Dates to remember:
  - Assignment 2: due Tue, 2/24 at 9pm
  - Assignment 2 Quiz: Wed, 2/25 in lecture